Práctica de Programación Orientada a Objetos (POO)

Del enunciado al diagrama UML y al código C++

Cipriano Rivera Escobar

Fundamentos de Informática

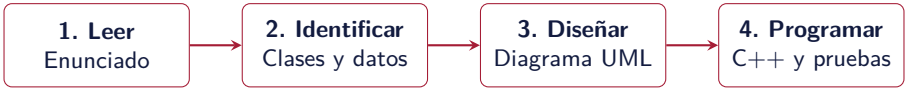
Universidad Nacional de Costa Rica

II Ciclo 2026

Práctica de análisis y diseño de clases

Ruta de aprendizaje

De un problema a una solución orientada a objetos



Indicaciones: lea cada situación y deduzca los atributos, tipos de datos, constructores y métodos necesarios. Primero entregue el UML; después programe en archivos `.h`, `.cpp` y `main.cpp`. Sin herencia, polimorfismo, arreglos, vectores ni `new`.

Ejercicio 1: Asociación de Maratón

Situación del problema

La Asociación Costarricense de Maratón prepara un informe para seleccionar a sus representantes en las próximas Olimpiadas. En la asociación participan varios corredores y, para cada uno, se conoce su nombre completo y su mejor marca expresada en segundos.

Antes de revisar a los participantes, la asociación registra el tiempo del récord latinoamericano, también expresado en segundos. Cada corredor debe recibir una evaluación individual para determinar si su marca está suficientemente cerca del récord.

La asociación también necesita conocer el promedio de los tiempos registrados y conservar la información del corredor que obtuvo la mejor marca de toda la jornada.

Clases a diseñar: `Corredor` y `AsociacionMaraton`.
Determine usted qué información y qué comportamientos corresponden a cada clase.

Ejercicio 1: Asociación de Maratón

Requerimientos del programa

1. Solicite el récord latinoamericano y la cantidad de participantes.
2. Registre, uno por uno, el nombre completo y el mejor tiempo de cada corredor. Utilice objetos de las dos clases para procesar la información.
3. Por cada participante, muestre su nombre, el tiempo en segundos y el equivalente en horas, minutos y segundos.
4. Indique si clasifica: la diferencia absoluta respecto al récord debe ser **menor al 5 %** de ese récord.
5. Al finalizar, muestre el tiempo promedio y el nombre y tiempo del corredor más rápido.

Condición técnica: la conversión de tiempo debe entregar horas, minutos y segundos mediante parámetros por referencia. Procese un corredor a la vez, sin arreglos ni vectores.

Ejercicio 1: trabajo por realizar

UML antes de escribir código

Parte A. Diagrama UML

- Identifique los atributos y sus tipos de datos en cada clase.
- Proponga constructores y métodos públicos apropiados.
- Incluya encapsulamiento, parámetros y tipos de retorno.
- Represente cómo una clase utiliza un objeto de la otra.

Parte B. Implementación

- Cree los archivos correspondientes a las dos clases.
- En `main.cpp`, solicite los datos, cree objetos y ejecute las operaciones.
- Pruebe el programa con varios corredores y diferentes tiempos.

Ejercicio 2: Análisis de números

Situación del problema

Un centro de apoyo académico necesita una herramienta sencilla para estudiar números enteros. El usuario ingresa tres valores en cualquier orden y el sistema debe organizarlos de menor a mayor.

Después, el sistema debe analizar dos intervalos: desde el número menor hasta el central, para identificar los números impares; y desde el central hasta el mayor, para identificar los números primos.

Para organizar la solución, cada número se representará como un objeto independiente. Otro objeto será responsable de trabajar con los tres números, ordenarlos y realizar el análisis solicitado.

Clases a diseñar: `NumeroEntero` y `AnalizadorNumeros`.
Deduzca los atributos y los métodos que necesita cada clase a partir del contexto.

Ejercicio 2: Análisis de números

Requerimientos del programa

1. Solicite tres valores enteros y cree un objeto para representar cada uno.
2. Cree el objeto que realizará el análisis utilizando los tres objetos anteriores.
3. Ordene los valores de forma ascendente, sin utilizar arreglos ni vectores.
4. Muestre los números impares entre el menor y el central, incluidos los extremos cuando corresponda.
5. Muestre los números primos entre el central y el mayor, incluidos los extremos cuando corresponda.

Ejemplo: si se ingresan `15, 3, 11`, el orden es `3, 11, 15`. Impares: `3, 5, 7, 9, 11`. Primos: `11, 13`. Un primo es mayor que 1 y solo es divisible entre 1 y sí mismo.

Ejercicio 2: trabajo por realizar

UML antes de escribir código

Parte A. Diagrama UML

- Determine qué información necesita conservar cada objeto.
- Identifique las operaciones de cada clase a partir del enunciado.
- Agregue constructores, visibilidad, parámetros y tipos de retorno.
- Muestre la relación entre ambas clases.

Parte B. Implementación

- Implemente ambas clases en sus archivos `.h` y `.cpp`.
- Desde `main.cpp`, cree los objetos y pruebe las operaciones.
- Verifique casos con valores repetidos, negativos y números menores que 2.

Entrega de la práctica

Productos solicitados

1. **Dos diagramas UML:** uno por ejercicio, cada uno con sus dos clases.
2. **Código C++:** archivos de encabezado, implementación y `main.cpp`.
3. **Pruebas:** ejemplos de entrada y salida que demuestren el funcionamiento.

El objetivo es aprender a **interpretar un enunciado**, identificar los elementos de cada clase y traducir el UML a código.

No se incluyen las soluciones.